

Practicle-4(A)

Aim: To create and implement a Java password strength checker using string methods or regular expressions to validate length, uppercase letters, digits, and special characters, and to classify passwords as Weak, Medium, or Strong based on the number of rules satisfied.

Code Screenshot:

A) PasswordChecker.java

```
public class PasswordChecker {  
  
    public static boolean checkLength(String pw) {  
        return pw.length() >= 8;  
    }  
  
    public static boolean checkUppercase(String pw) {  
        return pw.matches(".*[A-Z].*");  
    }  
  
    public static boolean checkDigit(String pw) {  
        return pw.matches(".*[0-9].*");  
    }  
  
    public static boolean checkSpecial(String pw) {  
        return pw.matches(".*[^a-zA-Z0-9].*");  
    }  
  
    public static String strength(String pw) {  
  
        int count = 0;  
  
        if (checkLength(pw))  
            count++;  
  
        if (checkUppercase(pw))  
            count++;  
  
        if (checkDigit(pw))  
            count++;  
  
        if (checkSpecial(pw))  
            count++;  
  
        if (count <= 1)  
            return "Weak";  
        else if (count <= 3)  
            return "Medium";  
        else  
            return "Strong";  
    }  
}
```

B) Driver.java

```
public class Driver {  
    Run | Debug | Run main | Debug main  
    public static void main(String[] args) {  
        String[] passwords = {  
            "abc",  
            "abcdefgh",  
            "Abcdefgh",  
            "abcdef12",  
            "Abcd1234!",  
            "Password1"  
        };  
        for (String pw : passwords) {  
            System.out.println("Password: " + pw);  
            System.out.println("Length >= 8: "  
                + PasswordChecker.checkLength(pw));  
            System.out.println("Uppercase: "  
                + PasswordChecker.checkUppercase(pw));  
            System.out.println("Digit: "  
                + PasswordChecker.checkDigit(pw));  
            System.out.println("Special Character: "  
                + PasswordChecker.checkSpecial(pw));  
            System.out.println("Strength: "  
                + PasswordChecker.strength(pw));  
            System.out.println(x: "-----");  
        }  
    }  
}
```

Output Screenshot:

```
PS C:\Users\Aryan\Documents\Charusat\Sem-3\Java\Practice-4\PassChecker> java Driver
Password: abc
Length >= 8: false
Uppercase: false
Digit: false
Special Character: false
Strength: Weak
-----
Password: abcdefgh
Length >= 8: true
Uppercase: false
Digit: false
Special Character: false
Strength: Weak
-----
Password: Abcdefgh
Length >= 8: true
Uppercase: true
Digit: false
Special Character: false
Strength: Medium
-----
Password: abcdef12
Length >= 8: true
Uppercase: false
Digit: true
Special Character: false
Strength: Medium
-----
Password: Abcd1234!
Length >= 8: true
Uppercase: true
Digit: true
Special Character: true
Strength: Strong
-----
Password: Password1
Length >= 8: true
Uppercase: true
Digit: true
Special Character: false
Strength: Medium
-----
```

Practicle-4(B)

Aim: To develop a Java chat-log filtering program using arrays, user input, split(), loops, case-insensitive string searching, and StringBuilder to identify matching messages and generate a simple report.

Code Screenshot:

A) ChatFilter.java

```
public class ChatFilter {  
  
    public static String filterLogs(String[] logs, String keyword) {  
        int count = 0;  
        StringBuilder report = new StringBuilder();  
  
        for (String line : logs) {  
            String[] parts = line.split(regex: " ", limit: 3);  
  
            if (parts.length < 3) {  
                continue;  
            }  
  
            String time = parts[0];  
            String user = parts[1];  
            String message = parts[2];  
  
            if (message.toLowerCase().contains(keyword.toLowerCase())) {  
                count++;  
                report.append(time)  
                    .append(str: " ")  
                    .append(user)  
                    .append(str: ": ")  
                    .append(message)  
                    .append(str: "\n");  
            }  
        }  
  
        return "Matches: " + count + "\n" + report;  
    }  
}
```

B) Drive.java

```

import java.util.Scanner;

public class Driver {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {

        String[] logs = {
            "10:05 alice Hello there",
            "10:10 bob How are you",
            "10:15"
        };

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter keyword: ");
        String keyword = sc.nextLine();

        String result = ChatFilter.filterLogs(logs, keyword);

        System.out.println(result);

        sc.close();
    }
}

```

Output Screenshot:

```

PS C:\Users\Aryan\Documents\Charusat\Sem-3\Java\Practicle-4\ChatFilterProgram> java Driver
Enter keyword: hello
Matches: 1
10:05 Aaryan: Hello there

PS C:\Users\Aryan\Documents\Charusat\Sem-3\Java\Practicle-4\ChatFilterProgram> java Driver
Enter keyword: aaryan
Matches: 2
10:05 Aaryan: Hello there
10:15 Aaryan: I'm good, thanks

PS C:\Users\Aryan\Documents\Charusat\Sem-3\Java\Practicle-4\ChatFilterProgram> java Driver
Enter keyword: thanks
Matches: 1
10:15 Aaryan: I'm good, thanks

PS C:\Users\Aryan\Documents\Charusat\Sem-3\Java\Practicle-4\ChatFilterProgram> java Driver
Enter keyword: welcome
Matches: 0

PS C:\Users\Aryan\Documents\Charusat\Sem-3\Java\Practicle-4\ChatFilterProgram> java Driver
Enter keyword: ramesh
Matches: 1
10:10 Ramesh: How are you

```

Learning Outcome: -

The learning outcome is to understand string handling and validation in Java using regular expressions and string methods. Students will learn to check password strength based on multiple conditions. They will practice arrays, loops, string splitting, and case-insensitive searching. They will also learn to use StringBuilder for creating reports efficiently. These programs improve problem-solving skills and practical Java programming knowledge.

GitHub Repositories: -

<https://github.com/d26it116/OOP/tree/159f27c0b1a369ac4529973d98446af6a77befb1/Practicle-4>